



Önálló laboratórium beszámoló

Távközlési és Mesterséges Intelligencia Tanszék

Készítette: Horváth Benedek
benedekhorvath@edu.bme.hu

Neptun-kód: D86EP7

Ágazat: mérnökinformatikus szak

Konzulens: Németh Felicián
nemethf@tmit.bme.hu

Konzulens: Lévai Tamás
levait@tmit.bme.hu

Téma címe: Médiaátvitel QUIC felett

Feladat:

A féléves munka célja a Media over QUIC (MoQ) protokollhoz tartozó moq-rs implementáció megfigyelhetőségi (observability) lehetőségeinek feltérképezése és elemzése. A feladat magában foglalja a különböző metrikagyűjtési és vizualizációs technológiák (pl. Prometheus, OpenTelemetry) összehasonlító vizsgálatát, valamint a moq-rs architektúrájához leginkább illeszkedő megoldás kiválasztását. A hallgatónak meg kell vizsgálnia, hogyan gyűjthetők hatékonyan rendszerszintű és alkalmazásszintű adatok elosztott környezetben.

Tanév: 2025/2026. tanév, I. félév

Tartalomjegyzék

1. Bevezetés	1
1.1. MoQ-hálózatok	1
1.2. Célkitűzés	1
2. Háttér	2
2.1. Adatgyűjtési módszerek	2
2.2. A MoQ ökoszisztéma	2
2.3. Használt technológiák	3
2.3.1. Prometheus vs. OpenTelemetry	3
2.3.2. Rust könyvtárak	3
3. Architektúra és tervezés	3
3.1. A moq-rs ökoszisztéma	4
3.2. A moq-metrics komponens	4
3.3. Adatgyűjtési stratégiák	5
3.3.1. A „QUIC-loop dilemma”	5
3.3.2. Metrikák kiválasztása	5
4. Megvalósítás	6
4.1. Metrikatípusok	6
4.2. Címkézés	7
4.3. Tervezési minták	7
4.4. Használat	8
4.4.1. Relay	8
4.4.2. Publisher és Subscriber	8
4.5. Fejlesztési nehézségek és tapasztalatok	8
5. Vizualizáció és elemzés	9
5.1. Grafana dashboard tervezése	9
5.2. Eredmények értelmezése	9
6. Fejlesztési lehetőségek és összegzés	11
6.1. QUIC-dilemma	11
6.2. Fejlett metrikák	11
6.3. Dockeresítés	11
6.4. Upstreaming	11
6.5. E2E mérések	11
6.6. Összegzés	11
Irodalomjegyzék	12

1. Bevezetés

A modern videóstreaming felhasználói egyre magasabb elvárásokat támasztanak a tartalom minőségével és valós idejű elérhetőségével szemben. Az olyan interaktív alkalmazásoknál, mint az élő közvetítések, a hagyományos protokollok késleltetése már zavaró lehet – például amikor a szomszéd előbb ünnepli a gólt, mint ahogy mi látjuk. Erre kínál megoldást a Media over QUIC (MoQ) [1], amely a QUIC protokoll [2] gyorsaságát és rugalmasságát használja ki a médiaátvitelre, felváltva az elavult TCP-alapú megoldásokat.

1.1. MoQ-hálózatok

A Media over QUIC nem csupán egy új szállítási protokoll, hanem egy teljesen új hálózati architektúrát is feltételez. A hagyományos, hierarchikus CDN (Content Delivery Network) struktúrák mellett megjelennek a dinamikusabb, relay-alapú topológiák. Ezekben a hálózatokban a csomópontok (relayek) nem csak egyszerűen gyorsítótáraznak, hanem aktívan részt vesznek az útvonalválasztásban és a tartalom elosztásában. Egy ilyen elosztott rendszerben a tartalom útja a hálózat állapota, a torlódások és a kliensek igényei alapján folyamatosan változhat. Ez a dinamikus útvonalválasztás azonban új kihívásokat is szül: hogyan biztosítható a QoS (Quality of Service), ha az útvonalak folyamatosan változnak?

1.2. Célkitűzés

A hatékony médiaátvitelhez elengedhetetlen a pontos rálátás a belső működésre. Jelenleg a MoQ implementációk megfigyelhetősége (observability) korlátozott, ami nehezíti, hogy jól felhasználható statisztikákat készítsünk. A feladat célja a moq-rs [3] implementáció mérési lehetőségeinek részletes vizsgálata. A munka során:

- Feltérképezésre kerülnek a lehetséges mérési pontok a moq-rs architektúrájában.
- Összehasonlításra kerülnek a különböző telemetriagyűjtési technológiák.
- Tervezésre és implementálásra kerül egy moduláris mérési rendszer.
- Az eredmények vizualizációja is megtörténik, hogy azok az üzemeltetők számára is értelmezhetők legyenek.

2. Háttér

A megfigyelhetőség (observability) a modern elosztott rendszerek üzemeltetésének alapköve. Három fő módja a logging, a tracing és a metrikagyűjtés. Jelen munka fókuszában a metrikák állnak. Azért a metrikagyűjtés került kiválasztásra, nem mondjuk a tracing, mert egy relay másodpercenként több ezer csomagot továbbíthat, és a tracing minden egyes csomagot nyomon követne, ami jelentős CPU- és memóriaterhelést jelentene. A metrikáknál pl. egy számláló növelése sokkal barátságosabb a rendszerre nézve. Továbbá ez a megoldás tökéletes alkalmas egy globális kép kialakítására, nincs szükség arra, hogy minden egyes csomagról tudjunk.

2.1. Adatgyűjtési módszerek

A metrikák gyűjtésének két alapvető modellje létezik: a **pull** és a **push**.

A **pull modell** (pl. Prometheus [4]) esetében a monitorozó szerver periodikusan lekérdezi („scrape”) az alkalmazás végpontját. Előnye, hogy központi backend gyűjti az adatokat, viszont nehézkes lehet a hálózat belüli – vagy különösen az azon kívüli – kommunikáció.

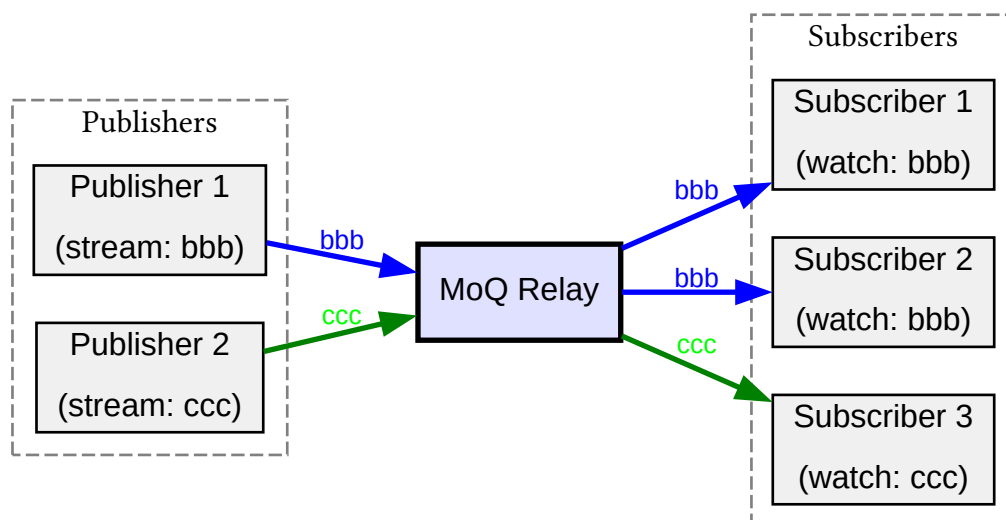
A **push modell** (pl. Graphite, OpenTelemetry OTLP [5]) esetében az alkalmazás küldi el az adatokat egy központi gyűjtőnek. Ez egyszerűbb hálózati kommunikációt jelent, mert nem kell tudnia a backendnek a külön megfigyelendő végpontokról, de emiatt nehezebben is kiszámítható a terhelés, könnyen alakulhat ki túlterhelés.

A moq-rs esetében a push modell ideálisabbnak tűnhet a hálózat dinamikus jellege miatt, de a Prometheus elterjedtsége miatt egyszerűbb és karbantarthatóbb a pull modell.

2.2. A MoQ ökoszisztéma

A Media over QUIC (MoQ) architektúra alapvetően három szereplőt különböztet meg:

1. **Publisher:** A tartalom előállítója (stream forrása, kamera stb.).
2. **Relay:** A közvetítő szerver, amely gyorsítótárazza és továbbítja az adatokat. Ezek alkotják a CDN (Content Delivery Network) gerincét.
3. **Subscriber:** A fogyasztó (pl. videólejátszó).



1. ábra: A Media over QUIC (MoQ) architektúra logikai felépítése több stream esetén.

2.3. Használt technológiák

2.3.1. Prometheus vs. OpenTelemetry

A **Prometheus** jelenleg az ipari standard a metrikák tárolására és lekérdezésére. Nagyon elterjedt, nagy ökoszisztémával rendelkezik, amit egy erős lekérdezési nyelv, a PromQL támogat.

Az **OpenTelemetry (OTel)** egy újabb szabvány, amely egyesíti a traceek, logok és metrikák gyűjtését, azonban használata bonyolult és körülményes. Előnye, hogy a backend (ami akár Prometheus is lehet) könnyen cserélhető.

Ezek miatt a Prometheus lett használva közvetlenül, mert egyrészt egyszerűbb a használata, másrészt kevesebb erőforrást is igényel, valamint most kifejezetten a metrikákon van a hangsúly.

A két technológia összehasonlítását az 1. táblázat tartalmazza.

Tulajdonság	Prometheus	OpenTelemetry
Modell	Pull (alapértelmezett)	Push (OTLP)
Adattípusok	Metrikák	Trace, Log, Metrika
Komplexitás	Alacsony	Magas
Rust támogatás	Kiváló	Fejlődő

1. táblázat: A Prometheus és az OpenTelemetry összehasonlítása.

2.3.2. Rust könyvtárak

A Rust ökoszisztémában a mérésre több megoldás létezik:

- **prometheus-client**: A Prometheus projekt hivatalos Rust klienskönyvtára [6]. Ez nem egy generikus facade, hanem egy típusbiztos, közvetlen implementáció, amely saját regisztert (Registry) kezel. Előnye a szoros integráció a Prometheus adatmodellel és a nagy teljesítmény.
- **metrics crate**: Egy népszerű, generikus könyvtár [7]. Bár rugalmas, a prometheus-client közvetlenebb kontrollt biztosít a metrikák felett.
- **opentelemetry crate**: Az OTel teljes implementációja [5]. Bár nagy tudású, nehezebb a használata és nagyobb overheaddel jár.

A moq-rs fejlesztése során a prometheus-client crate volt használva, mivel ez biztosítja a legstabilabb és leghatékonyabb módot a Prometheus formátumú adatok exportálására.

3. Architektúra és tervezés

A rendszer tervezésekor az elsődleges szempont a modularitás és a minimális overhead volt. Kiemelt cél volt, hogy az architektúra minél függetlenebb legyen az eredeti moq-rs kódtól, elősegítve ezzel a későbbi upstreamelést (PR) egy egyszerűsített demó változat formájában. A méréseket végző kód ezért egy különálló könyvtárba, a moq-metrics-be került kiszervezésre.

3.1. A moq-rs ökoszisztéma

A moq-rs projekt több, egymásra épülő komponensből áll, amelyek mindegyike egy-egy specifikus feladatot lát el a médiaátviteli láncban. A moq-metrics könyvtár célja, hogy egységes mérési felületet biztosítson ezekhez a komponensekhez.

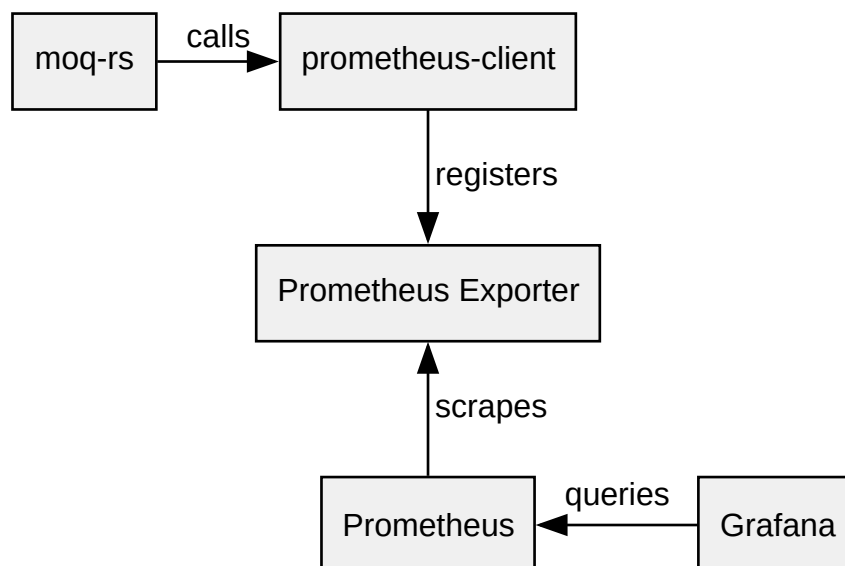
- **moq-transport:** A rendszer magja, amely a MoQ protokoll alapvető alkotóelemeit (objektumok, trackek, subscriptionök) valósítja meg a QUIC felett. Ez a réteg felelős a kapcsolatkezelésért és az adatfolyamok feldolgozásáért.
- **moq-relay:** A központi médiatovábbító szerver. Feladata a publikálók és feliratkozók közötti kapcsolatok kezelése, valamint az adatfolyamok gyorsítótárazása és továbbítása. Ez a komponens a legkritikusabb a teljesítmény szempontjából, így itt a legfontosabb a részletes monitorozás.
- **moq-pub és moq-sub:** A beépített, referencia MoQ-kliens implementációk. A moq-pub médiát vagy élő streamet publikál, míg a moq-sub ezeket fogadja és játssza le.

A moq-metrics egy lazán kapcsolódó crate-ként illeszkedik a könyvtárba. Nem része a protokollnak (egyelőre), de a moq-transport, a moq-relay és a kliensek is használják metrikák gyűjtésére. Ez a kialakítás lehetővé teszi, hogy a mérési logika elkülönüljön az üzleti logikától, ugyanakkor minden komponens egységes formátumban szolgáltatson adatot.

3.2. A moq-metrics komponens

Ez a komponens felelős a metrikák regisztrálásáért és az exporterek kezeléséért. Gyakorlatilag a Prometheus backendet köti össze az alkalmazás többirészeivel. Főbb feladatai:

- a prometheus-client használata,
- a Prometheus exporter elindítása egy konfigurálható HTTP porton,
- hálózati metrikák gyűjtése,
- rendszermetrikák (CPU, memória) periodikus gyűjtése.



2. ábra: A tervezett observer pipeline.

3.3. Adatgyűjtési stratégiák

A projektben a **pull modell** került kiválasztásra a Prometheus széleskörű támogatottsága miatt. Minden `moq-rs` komponens (relay, publisher, subscriber) elindít egy HTTP szervert, ahonnan a Prometheus szerver „scrapelheti” az aktuális metrikákat.

3.3.1. A „QUIC-loop dilemma”

A fejlesztés egyik központi kérdése az volt, hogy mikor és hogyan frissüljenek a metrikák értékei. Két eltérő megközelítést is alkalmazásra került a metrikák természetétől függően:

1. **QUIC metrikák:** A QUIC kapcsolatok állapota (pl. aktív kapcsolatok száma, RTT) a `moq-transport` belső memóriájában folyamatosan rendelkezésre áll. Felesleges lenne egy külön szálon, másodpercenként lekérdezni és másolni ezeket az adatokat, ha a Prometheus csak 15 másodpercenként kéri le őket. Ezért itt a „lazy” kiértékelés történik: a metrikák frissítése csak akkor történik meg, amikor beérkezik a HTTP GET kérés a `/metrics` végpontra. Ezt a HTTP szerver handlerében lett megvalósítva, amely közvetlenül a válaszadás előtt olvassa ki az aktuális állapotot. Ez minimalizálja a CPU terhelést és a kontextusváltásokat.
2. **Rendszermetrikák:** A rendszermetrikák, különösen a CPU terhelés, gyorsan változnak. Ha itt is on-demand módon történne a lekérés, és csak a scrape pillanatában (pl. 5 másodpercenként) lenne mérve a terhelés, könnyen lemaradhatna egy-egy kicsúcsosodás. Ezért a `sysinfo` [8] alapú méréshez egy dedikált, háttérben futó `tokio::task` indul (`tokio::spawn` [9]). Ez a ciklus sűrűbben mintavételez, és egy belső állapotban tárolja az aggregált értékeket. Amikor a Prometheus lekérdez, már ezt a pontosabb, simított értéket kapja meg. Bár ez némi plusz erőforrást igényel, elengedhetetlen a valós teljesítményképhez.

Ez a hibrid megoldás biztosítja az egyensúlyt, a lassabban változó hálózati metrikák és a gyorsabban változó rendszermetrikák így optimálisan kerülnek kiértékelésre.

3.3.2. Metrikák kiválasztása

Komoly fejtörést okozott a megfelelő metrikák kiválasztása. Nem az volt a cél, hogy „minden legyen mérve”, hanem hogy olyan mutatókat lehessen mérni, amelyekből következtetni lehet a hálózat minőségére és a rendszer korlátaira. A `lib.rs`-ben definiált metrikák négy fő kategóriába sorolhatók:

1. **Forgalmi adatok:** Az átvitt adatmennyiség (bájtok) és objektumok száma, mind a publikálók, mind a feliratkozók irányába. Ez ad képet a rendszer tényleges terheléséről.
2. **Kapcsolatok állapota:** Az aktív QUIC kapcsolatok, publikálók és feliratkozók száma, valamint a hálózati minőség mutatói (pl. RTT, csomagvesztés).
3. **MoQ specifikus metrikák:** A protokoll szintű entitások, mint a meghirdetett és feliratkozott sávok követése.
4. **Rendszererőforrások:** A CPU és memória felhasználása mind folyamat, mind rendszer-szinten, ami kritikus a skálázódás és a hardveres korlátok felismerése szempontjából.

4. Megvalósítás

A megvalósítás során a prometheus-client crate API-ja került felhasználásra a megfelelő mérési pontokon a moq-rs kódbázisában.

4.1. Metrikatípusok

A lib.rs számos metrikát definiál, amelyek közül a teljesség igénye nélkül néhány fontosabb példa:

- **moq_relay_bytes_received_from_publisher (Counter):** A publikálótól fogadott adat-mennyiség bájtokban.
- **moq_relay_quic_connections_active (Gauge):** Az éppen élő QUIC kapcsolatok száma.
- **moq_relay_active_subscribed_tracks (Gauge):** Az aktív feliratkozások száma névtér (namespace) szerint bontva.
- **moq_relay_quic_rtt_milliseconds (Gauge):** A QUIC kapcsolatok körkörös késleltetése (RTT) milliszekundumban.
- **process_cpu_usage_percent (Gauge):** A folyamat CPU használata százalékban.

Példa a moq-metrics/src/lib.rs-ből, ahol egy Rust makró segítségével kerültek definiálásra a metrikák. Ez a megoldás jelentősen csökkenti a kódismétlést és egyszerűsíti új metrikák felvételét:

```
// A makró használata: itt soroljuk fel az összes metrikát
define_metrics! {
    objects_sent:
        Counter<u64>,
        "moq_relay_objects_sent",
        "Total objects sent",
    // ... további metrikák ...
}
```

A fenti makróhívás a fordítási időben a következő (egyszerűsített) kódra bomlik ki, ami jól szemlélteti, mennyi „boilerplate” kódot spóroltunk meg:

```
// A generált kód (kibontva)
#[derive(Clone)]
pub struct MoqMetrics {
    pub objects_sent: Counter<u64>,
    // ... többi mező
}

impl MoqMetrics {
    pub fn new(registry: &mut Registry) -> Self {
        let objects_sent = <Counter<u64>>::default();
        registry.register(
            "moq_relay_objects_sent",
            "Total objects sent",
            objects_sent.clone()
        );
        // ... többi regisztráció

        Self {
```



```
        objects_sent,  
        // ... többi mező  
    }  
}  
}
```

A metrikák növelése pedig a kód megfelelő pontjain történik, például a `moq-transport/src/session/subscriber.rs`-ben, amikor adat érkezik:

```
// Példa a számláló növelésére (subscriber.rs)  
pub fn add_bytes_received(count: u64) {  
    GLOBAL_METRICS  
        .metrics  
        .bytes_received_from_publisher  
        .inc_by(count);  
}  
  
// ...  
// A fogadott adat méretének hozzáadása a metrikához  
moq_metrics::add_bytes_received(data.len() as u64);
```

4.2. Címkézés

A metrikák nem csupán egyszerű számlálók, címkékkel is el lehet őket látni. Például a sávszélesség mérésekor a namespace címke jelöli, hogy melyik streamhez tartozik az adat. Ez teszi lehetővé a Grafanában a későbbi szűrést és a részletes bontást (pl. melyik stream „eszi” a legtöbb sávszélességet, vagy melyik relay van földrajzilag leterhelve), ami sokat segít a mutatók összeállításában. Persze a jelenleg implementált címkéken kívül igény szerint lehetne még implementálni, pl. track ID, IP, port stb., ami hasznos lehet.

4.3. Tervezési minták

A fejlesztés során két fontos tervezési minta került alkalmazásra a kód egyszerűsítése és a biztonság növelése érdekében:

1. **Singleton:** Bár a globális változók használata általában kerülendő, ebben az esetben a `GLOBAL_METRICS` singleton (a `once_cell::sync::Lazy` [10] segítségével) jelentősen egyszerűsítette a hívásokat. Így nem kellett minden függvénynek paraméterként átadni a metrika registryt, hanem bárhol elérhetővé váltak a számlálók (pl. `moq_metrics::increment_active_connections()`).
2. **RAII:** A kapcsolatok és trackek számának mérésekor fontos, hogy a számlálók értéke csökkenjen, amikor az erőforrás felszabadul. A manuális `dec()` hívások helyett RAII (Resource Acquisition Is Initialization) guardokat lettek implementálva. Ezek olyan structok, amelyek `Drop` implementációja automatikusan csökkenti a metrikát, amikor a változó kikerül a scopeból.
3. **Mutex optimalizáció:** Kezdetben minden metrika Mutex mögé került a szálbiztonság érdekében. A munka közben azonban kiderült, hogy a `prometheus-client` primitívjei (`Counter`, `Gauge`) belső atomi műveleteket (`std::sync::atomic`) használnak, így önmagukban is szálbiztosak. A felesleges lockok eltávolítása egyszerűsítette a kódot és javította a teljesítményt.

4.4. Használat

A kód olvashatóságának megőrzése érdekében a mérések a logikától elválasztva kerültek elhelyezésre. A `lib.rs`-ben definiált globálisan elérhető publikus függvényekkel egyszerűen növelhető egy-egy számláló a megfelelő helyen.

4.4.1. Relay

A relayekben mérhető a legtöbb dolog, ugyanis a MoQ-hálózat gerincét ezek teszik ki. Itt történik a bejövő és kimenő forgalom mérése sessionönként. Itt számos metrika lett implementálva, mint a feliratkozott trackek száma, küldött objektumok, bájtok száma, aktív QUIC kapcsolatok száma, valamint persze a CPU és memóriával kapcsolatos metrikák.

4.4.2. Publisher és Subscriber

A végpontokon is a relayhez hasonló metrikák lettek implementálva, de inkább a hálózat minőségére koncentrálva, hogy lehessen E2E metrikákat mérni.

4.5. Fejlesztési nehézségek és tapasztalatok

A fejlesztés során több kihívás is felmerült:

- Mérési pontok helyes kiválasztása:** A leggyakoribb hiba a „dupla számolás” volt. Mivel bizonyos függvények a kód több pontjáról vagy többször is meghívódhatnak egy esemény bekövetkezetekor, nehéz volt megtalálni azt az egyetlen pontot, ahol a metrikát biztonságosan lehet növelni. Emiatt például az aktív feliratkozott trackekre vonatkozó metrika eleve a feliratkozókra vonatkozott, de kiderült, hogy mindig trackenként lehet csak növelni, és egy streamhez alapértelmezetten nem is egy, hanem több is tartozik, mert mindig van egy „init” track a rendes médiafolyam mellett.
- A QUIC-loop dilemma:** A korábban említett „on-demand” mérésnek van egy hátránya. A quinn könyvtárban [11] a kapcsolatok statisztikáinak lekérdezése (`conn.stats()`) belső zárolással (lock) jár. Ha hirtelen megnő a QUIC kapcsolatok száma (pl. több ezer kliens), a Prometheus lekérdezés (scrape) pillanatában a rendszernek egyszerre kell iterálnia az összes kapcsolaton. Ez rövid ideig tartó, de intenzív blokkolást okozhat, ami extrém esetben akadályozhatja a folyamatban lévő QUIC kommunikációt (pl. új kapcsolatok fogadását). Ez a projekt esetében ez természetesen nem okoz problémát, de a jövőben figyelmet igényel.
- Rejtett hibák:** A metrikák bevezetése segített beazonosítani egy vélt implementációs hibát. A `moq_relay_bytes_received_from_publisher` számláló növekedéséből látszott, hogy a relay akkor is fogadott adatot a publishertől, amikor már nem volt aktív feliratkozó. Ez egy olyan hibát jelzett, ami valószínűleg olyan helyzetben történik, ahol a DROP esemény nem került megfelelően átadásra a publisher felé. Ez az eset is mutatja, hogy a metrikagyűjtés valóban hasznosnak bizonyul.

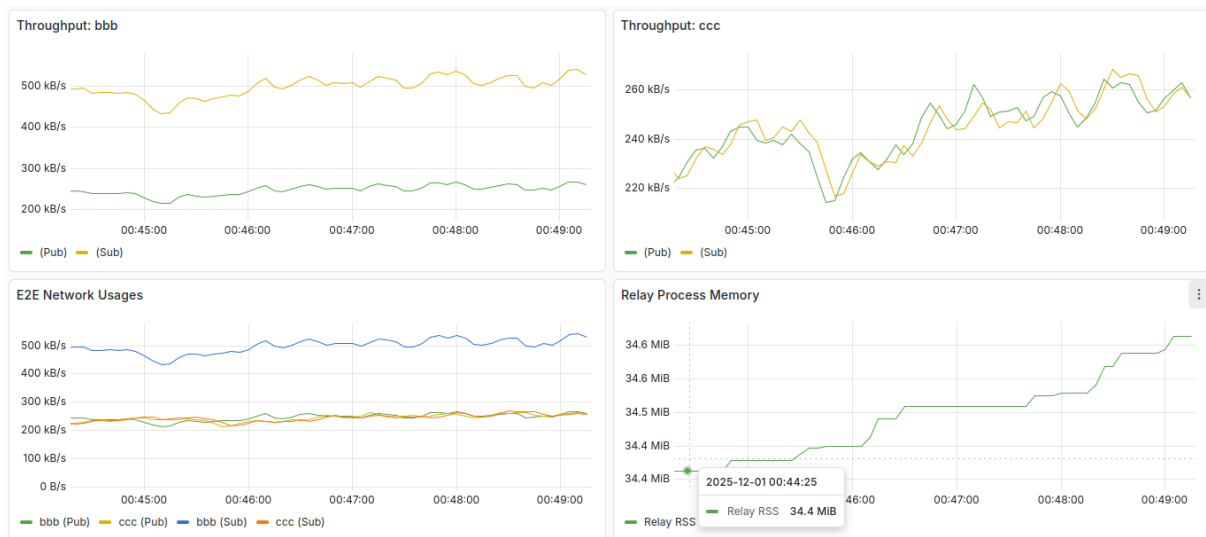
5. Vizualizáció és elemzés

A begyűjtött metrikák megjelenítésére a Grafana [12] került kiválasztásra, mivel natívan támogatja a Prometheus és rugalmasan testreszabható.

5.1. Grafana dashboard tervezése

A dashboard úgy lett kialakítva, hogy a legfontosabb vagy legértelmesebb mutatókat lehessen demózni. A grafana/moq-rs-*.json fájl alapján a tömkek között a következő panelek kerültek megvalósításra:

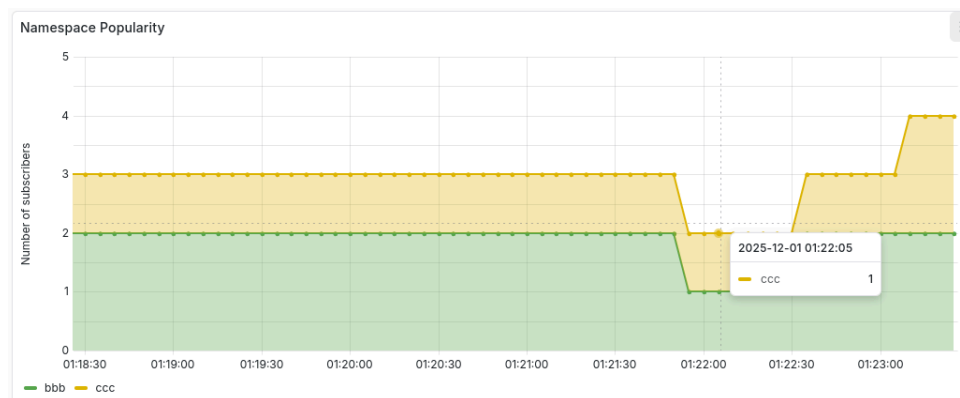
- **Throughput & Network Usage:** A publikálók által küldött és a feliratkozók által fogadott adatmennyiség valós idejű grafikonja. Itt látható a rendszer teljes sávszélességigénye.
- **Streamsintű sávszélességbontás:** A forgalmat streamekre lebontva is nyomon lehet követni egy grafikonon.
- **Relay & System Resource Usage:** A relay folyamat és a teljes rendszer CPU és memória használata. Külön grafikon mutatja a folyamat és a rendszer memórafogyasztását.
- **Track Sync:** A közzé tett és a feliratkozott sávok (tracks) számának szinkronja, ami segít a „szivárgó” feliratkozások felfedezésében.



3. ábra: A megvalósított Grafana dashboard egy részlete.

5.2. Eredmények értelmezése

A mérések során a rendszer erőforrásigényét és a hálózati stabilitást vizsgáltuk. Látszódik, ha változik a feliratkozók vagy a publisherek száma, új stream jön be a képbe, és az is látszódik, hogy hogyan változik a terhelés ezek függvényében. Jól látszódik az is, ha valamelyik stream a szokásosnál több erőforrást használ, megnő a sávszélesség, hirtelen változás esetén a késleltetés is növekedik.



4. ábra: A különböző streamek használatának megoszlása.

6. Fejlesztési lehetőségek és összegzés

A féléves munka során sikeresen implementálásra került egy alapvető mérési modul a moq-rs-ben, de számos területen van még lehetőség a fejlődésre.

6.1. QUIC-dilemma

A korábban már említett QUIC-metrikák mérését meg kell vizsgálni, és alternatívát találni.

6.2. Fejlett metrikák

A jelenlegi metrikapalettát még lehet bővíteni igény szerint, például a puffer és cache megfigyelésével, egyéb, lokáció alapú metrikákkal, stb.

6.3. Dockeresítés

A tesztelés és a reprodukálhatóság érdekében hasznos lenne a teljes környezetet (relay, publisher, subscriber, Prometheus, Grafana) „dockeresíteni”. Ez nagyban megkönnyítené a rendszer kipróbálását és a fejlesztést.

6.4. Upstreaming

A fejlesztett moq-metrics könyvtár jelenleg a saját forkban él. A hosszú távú cél, hogy a tapasztalatok alapján egy letisztult, egyszerűsített demó változat készüljön, amely Pull Request formájában visszakerülhet az eredeti moq-rs projektbe. Ezáltal a közösség számára is elérhetővé válna egy alapvető, de bővíthető megfigyelhetőségi réteg.

6.5. E2E mérések

Vannak olyan mérendő adatok, amik nem nyerhetők ki ezzel az egyszerű, tiszta Prometheus implementációval. Érdemes lehet megvizsgálni a lehetőségét annak, hogy az egyes objektumokhoz időbélyeget fűzzünk hozzá, amivel mérhetővé válna az E2E késleltetés mérése. Erre létezik is már formátum a MoQ draftjában, amit lehetne használni, de még nincs implementálva.

6.6. Összegzés

A laboratóriumi munka során áttekintésre kerültek a Media over QUIC technológia kihívásai és a modern megfigyelhetőségi eszközök. Tervezésre és megvalósításra került egy moduláris mérési könyvtár, amely képes valós időben információt szolgáltatni a rendszer működéséről. A mérések eredményei igazolták, hogy lehetséges a metrikák gyűjtése MoQ-környezetben is, és van értelme egy rendes implementációnak.

Irodalomjegyzék

- [1] L. Curley, K. Pugin, és S. Nandakumar, „Media over QUIC Transport”, Internet-Draft, 2024. [Online]. Elérhető: <https://datatracker.ietf.org/doc/draft-ietf-moq-transport/>
- [2] J. Iyengar és M. Thomson, „QUIC: A UDP-Based Multiplexed and Secure Transport”, RFC RFC9000, máj. 2021. [Online]. Elérhető: <https://www.rfc-editor.org/rfc/rfc9000.html>
- [3] Cloudflare, „moq-rs Repository”. [Online]. Elérhető: <https://github.com/cloudflare/moq-rs>
- [4] The Prometheus Authors, „Prometheus Documentation”. 2024. [Online]. Elérhető: <https://prometheus.io/docs/introduction/overview/>
- [5] OpenTelemetry Authors, „OpenTelemetry Documentation”. 2024. [Online]. Elérhető: <https://opentelemetry.io/docs/>
- [6] Prometheus Team, „prometheus-client”. [Online]. Elérhető: https://docs.rs/prometheus-client/latest/prometheus_client/
- [7] T. Lawrence, „metrics”. [Online]. Elérhető: <https://docs.rs/metrics/latest/metrics/>
- [8] G. Gomez, „sysinfo”. [Online]. Elérhető: <https://docs.rs/sysinfo/latest/sysinfo/>
- [9] Tokio Contributors, „Tokio Runtime”. [Online]. Elérhető: <https://tokio.rs/>
- [10] A. Kladov, „once_cell”. [Online]. Elérhető: https://docs.rs/once_cell/latest/once_cell/
- [11] Quinn Contributors, „quinn”. [Online]. Elérhető: <https://github.com/quinn-rs/quinn>
- [12] Grafana Labs, „Grafana Documentation”. 2024. [Online]. Elérhető: <https://grafana.com/docs/>